

Project Report: Municipal Regulation Parsing Engine (Phase 1)

1. Project Scope, Purpose, and Technology Stack

Purpose

The objective is to engineer an automatic platform to internalize unstructured municipal rules (on websites, PDFs, etc.) and convert them to a universal and machine-readable format. This will allow better search, comparison, and review of regulations in various municipalities.

Phase 1 Scope

The initial stage is aimed at the construction and deployment of a Minimum Viable Product (MVP) to a development environment. The core deliverables are:

- A functional **scraping module** to retrieve documents from a target municipality.
- A **parsing module** that can correctly divide a document into its constituent parts (sections, tables, images, lists).
- A **QA test suite** to validate the output of the parser.
- Deployment of the system on Google Cloud Platform (GCP).
- **Out of Scope:** Artificial Intelligence for interpretation or analysis is not a focus for this phase. The methodology will be strictly **Test-Driven Development (TDD)**.

Technology Stack

- **Language:** Python 3.11+
 - **Cloud Platform:** Google Cloud Platform (GCP)
 - **Architecture:** A serverless, event-driven architecture.
 - **Orchestration & Compute:** **Cloud Functions** for triggering scrapers and **Cloud Run** for the parsing service API.
 - **Storage:** **Google Cloud Storage (GCS)** for storing raw (PDF/HTML) and processed (JSON) documents.
 - **Database:** **Firestore** or **Cloud SQL** (PostgreSQL) for storing metadata and structured results.
-

2. Chosen Role: Parsing Team Engineer

I will be on the **Parsing Team**. Although the scraping team's task of navigating changing websites is not easy, the transformative process of documents is the intellectual property of this project, and the most complicated logic. I will work on the transformation of a raw file (a 300-page PDF by-law) into a structured and hierarchical data object.

3. Key Tasks and Challenges (Parsing)

Tasks

1. **Document Analysis:** Examine sample regulations of a pilot municipality (Laval) to find overall structuring patterns (e.g., "Article X.X.X" or minuscule "Tableau Y" and numbered lists).
2. **Algorithm Development:** Develop heuristic and rule-based algorithms to identify and extract titles, paragraphs, tables, lists, and images in PDFs and HTML files.
3. **Standardization:** Specification of a common JSON data format of the `ParsedDocument` output. The flexible nature of this model needs to represent any regulatory document.
4. **Content Extraction:** Logic to handle both text-based PDFs (extracting text directly) and scanned/image-based PDFs (integrating an OCR engine).
5. **Testing:** Write unit and integration tests for the parsing module according to our TDD approach. This involves developing tests for edge cases such as complex tables, malformed documents, and nested lists.
6. **API Development:** Encapsulate parsing logic in a Dockerized REST API deployed on Cloud Run, it will accept a document location and return the structured JSON.

Challenges

- **Poor Standardization:** This is the main issue. Documents are not standardized in their format. Headings could be stipulated on font size, bold, or using keywords. The solution should be tolerant of this variance.
 - **Scanned PDFs:** Low-quality scans will produce errors during OCR, requiring cleanup and validation logic.
 - **Complex Data Structures:** Parsing multi-page tables, or tables containing merged cells, will be challenging and may need specialized logic.
 - **Hierarchical Sections:** Correctly identifying the parent-child relationships in sections (e.g., Section 1 -> Article 1.1 -> Paragraph 1.1(a)) is critical.
-

4. Domain Model: The Parsing Module

The architecture is designed around a reusable parsing library and city-specific plug-ins, which promote extensibility.

Core Reusable Components

- `ParserOrchestrator` : The primary service class. It takes a request that includes a document and a city identifier. It loads the correct `CityPlugin` with the city identifier.
- `IParserStrategy` (Interface): This is an abstract class, defines a `parse(document)` method. We'll have concrete implementations, such as `PDFParserStrategy` and `HTMLParserStrategy` .
- `StructureExtractor` : A utility module with stateless functions for common operations, like `find_tables()` , `extract_lists()` , and `identify_headings_by_font()` .
- `ParsedDocument` (Data Model): A class that represents the final output. It will be a tree-like structure consisting of `Section` , `Table` , and `Image` objects (may include others).

City-Specific Plug-in

- `CityPlugin` (e.g., `LavalPlugin`): This is the key to our plug-in system. It is a configuration class that informs the `ParserOrchestrator` about how to parse documents of a certain city. It defines:
 - Primary document type (e.g., PDF).
 - Regex patterns are applied to identify by-law numbers or section titles, specific to the city.
 - Heuristics, including the font size or the name of an H1, H2, and H3 title in the fonts in the document pages of that city.

Python

```
# Simplified Conceptual Code
class ParserOrchestrator:
    def process_document(self, doc_path: str, city_plugin: CityPlugin):
        # 1. Use the plugin to get the right parsing strategy
        parser_strategy = city_plugin.get_parser_strategy()

        # 2. The strategy uses core tools + plugin-specific rules
        parsed_doc = parser_strategy.parse(doc_path,
rules=city_plugin.get_rules())
        return parsed_doc

class CityPlugin:
    def get_parser_strategy(self) -> IParserStrategy: ...
    def get_rules(self) -> dict: ...
```

```

class LavalPlugin(CityPlugin):
    def get_parser_strategy(self):
        return PDFParserStrategy() # Laval primarily uses PDFs

    def get_rules(self):
        # Laval-specific logic goes here
        return {"heading_font_size_h1": 16, "section_regex":
r"Article\s\d+\.\d+"}

```

5. Gang of Four (GoF) Design Patterns

1. **Strategy Pattern:** This is the core of the domain model. The `ParserOrchestrator` is run with a specific `IParserStrategy` (`PDFParserStrategy`, `HTMLParserStrategy`). This allows us to change the parsing algorithm, depending on document type, without touching code in the orchestrator, and making it easy to introduce new document types (e.g., `DOCX`) in the future.
2. **Adapter Pattern:** The `CityPlugin` acts as an **Adapter**. It adapts the generic `ParserOrchestrator` to the interface of a particular (and, in many cases, quirky) set of document standards of a given municipality. It provides a translation layer between our generic tools and the unique formatting rules of the city.
3. **Composite Pattern:** This pattern will be used in the implementation of the `ParsedDocument`. A section object may contain a list of child objects. Children may be other section objects (subsections) or even leaf nodes directly, like `Table` or `Paragraph`. This enables the client to treat a section and the document uniformly, simplifying processing.

6. Technology and Library Research

Library/Tool	Purpose	License	Notes
PyMuPDF (Fitz)	High-performance PDF text/metadata extraction.	AGPL 3.0	Extremely fast and accurate for text-based PDFs. The AGPL license requires our service to be open-source if distributed. For internal use, this is fine.
pdfplumber	Robust table extraction from PDFs.	MIT	Built on top of <code>pdfminer.six</code> . Excellent for identifying table boundaries and extracting cell data. Can be used alongside PyMuPDF.

Library/Tool	Purpose	License	Notes
pytesseract	Python wrapper for Google's Tesseract OCR.	Apache 2.0	The go-to for OCR on scanned documents. The accuracy will depend on the quality of the Tesseract engine and image pre-processing.
Beautiful Soup 4	HTML parsing.	MIT	The standard for parsing HTML documents. Essential for the <code>HTMLParserStrategy</code> .

7. Risk Analysis and Mitigation

Risk	Probability	Impact	Mitigation Strategy
Extreme Document Format Variance	High	High	Mitigation: To create a benchmark, it is better to start with one city and examine its documents in-depth. Build the system to be rule-based and configurable via the <code>CityPlugin</code> from day one.
Poor Quality Scanned Documents	Medium	High	Mitigation: Create an image preprocessing pipeline (e.g., using OpenCV) to deskew, binarize, and clean up pages before sending them to OCR. Mark pages with low confidence as requiring manual QA.
Scope Creep ("Perfect Parsing")	High	Medium	Mitigation: Follow the action plan of TDD. Establish a clear "Definition of Done" with the QA team regarding parsing accuracy of the most important document elements (i.e., 95% accuracy in section titles).
Upstream Website Changes	High	Low	Mitigation: This primarily affects the scraping team, but by design, the plug-in architecture isolates scrapers. Implement monitoring and alerting for scraping failures.

8. Appendix: Cost Estimation and Scaling

This project is an excellent candidate for a serverless architecture, which keeps costs low and scales automatically.

Cost Breakdown (GCP)

- **Cloud Run (Parsing Service):** Priced per vCPU-second and request. With a free tier, initial development and testing costs will be **close to \$0**. A production workload processing 10,000 documents a month might cost **\$20-50/month**.
- **Cloud Functions (Scraping Trigger):** Also has a large free tier. Costs will be negligible, likely **<\$5/month**.
- **Cloud Storage:** Storing 50 GB of raw and processed documents would cost about **\$1/month**.
- **Potential Major Cost:** If we decide to use an OCR system like Google Cloud Vision API, rather than running Tesseract ourselves, the expenses might be greater. Vision API costs ~\$1.50 per 1,000 pages. Processing 10,000 pages would be ~\$15. This is a sacrifice between the operational cost and the development effort.

Scaling

The architecture is designed to scale horizontally.

- **Adding a New City:** Requires creating a new `CityPlugin` and a scraping configuration. It does not involve re-architecting the core system.
- **Load Management:** Cloud Run automatically scales to the requirements of incoming requests by increasing or decreasing the number of container instances. Assuming that we have 1,000 documents that have to be parsed at a time, Cloud Run can support the load (then accurately reduce back to zero) to keep the costs down to just what we use.
- **Cost Scaling:** Costs will scale linearly with the number of documents processed.